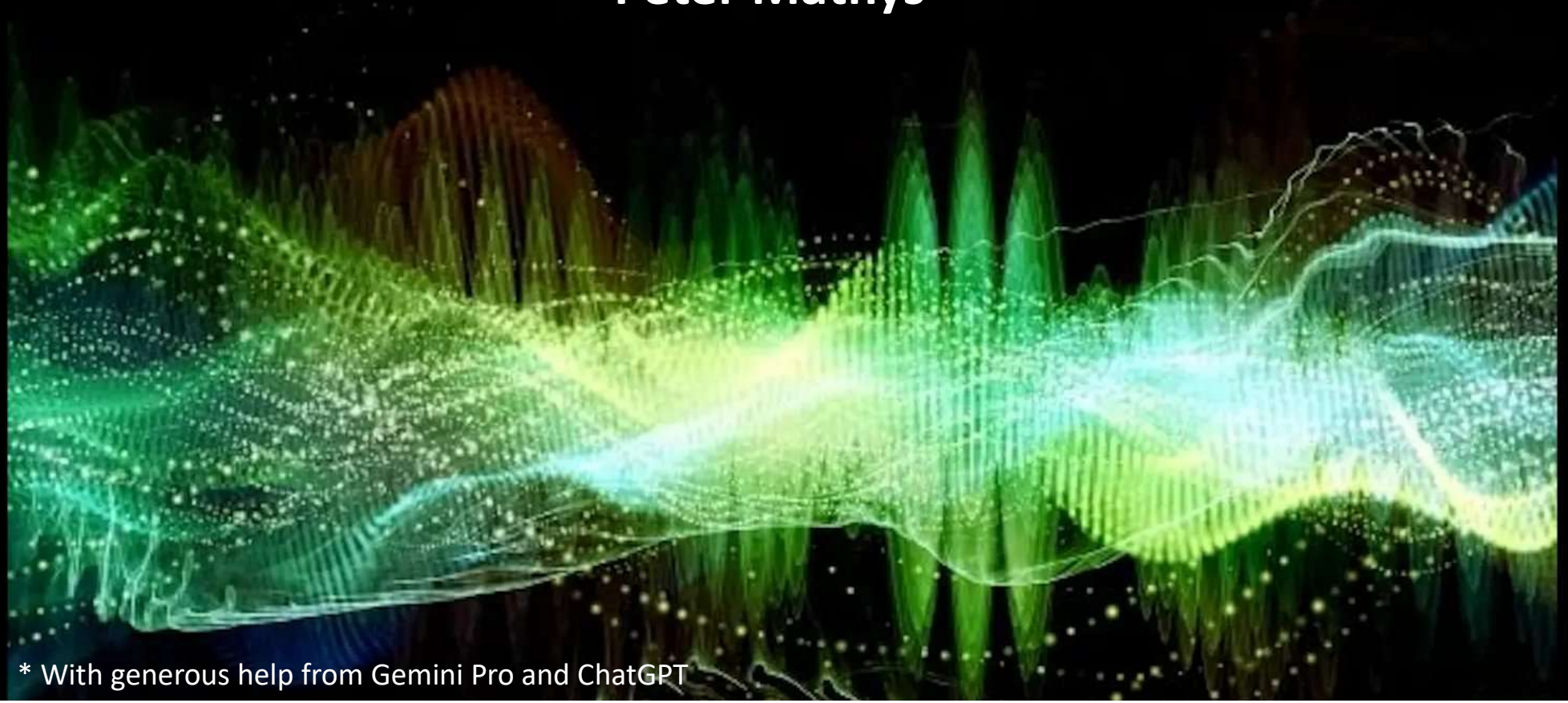
# ECEN 4/5632 Theory and Application of Digital Filters

## Lecture 26, 11-12-26

Peter Mathys*



* With generous help from Gemini Pro and ChatGPT

# Machine Learning Foundations

## Supervised Learning, Data Splits, and Scikit-Learn

Features X + labels y → fit a model → evaluate honestly → understand failure modes

WEEK 13 • LECTURE 26 • 75 MINUTES

# Learning outcomes

BY THE END OF LECTURE 26

00–20 MIN

- Write the supervised-learning problem as a mapping from feature vectors to labels.
- Explain the distinct roles of training, validation, and test data.
- Recognize source-level leakage in segmented audio datasets.
- Convert time-varying MFCC matrices into fixed-length vectors for classical models.
- Train and evaluate a baseline classifier using the scikit-learn estimator API and a confusion matrix.

Feature vector X to label y mapping

Train, Validate, Test datasets

Leakage between Train and Test datasets

Reduction of features Mel spectrogram → MFCC → mean/std MFCC

Train/evaluate of baseline classifier in Scikit-Learn

# From rules to learned mappings

Dimensions:
$X \in \mathbb{R}^{N \times D}$
$y \in \{1, 2, \ldots, C\}^N$

C: # of classes

SUPERVISED LEARNING CHANGES WHO WRITES THE DECISION RULE

00–20 MIN

**Traditional program**
Engineer writes rules
Input + rules → output
Useful when rules are explicit

fully engineered expert system

**Supervised ML**
Provide labeled examples
Algorithm estimates θ
Generalize to unseen examples

Supervised ML

**Audio example**
x = audio features
y = class label
ŷ = predicted class

Audio examples
X audio features
y class label
ŷ prediction

$$\hat{y} = f(x ; \theta)$$

$\hat{y} = f(X ; \theta)$ θ learning parameters

# Feature matrix X and target vector y

SCIKIT-LEARN USES A TABULAR CONTRACT

00–20 MIN

$$X \in R^{(N \times D)} \quad y \in \{1,\ldots,C\}^N$$

13-20 for MFCC

| N | D | C |
|---|---|---|
| number of clips | features per clip | number of classes |

N

vector dimension D

matrix $X[i,:] = \underline{x}_i$

- Every row X[i,:] must have the same length D.
- y[i] is the label paired with exactly that row.
- For 20-MFCC mean+std features, D = 40.

$(\underline{x}_1, y_1), (\underline{x}_2, y_2), \ldots, (\underline{x}_N, y_N)$

supervised learning N data samples for training

# What is a “feature”?

A COORDINATE CHOSEN TO MAKE PREDICTION EASIER

00–20 MIN

D is large

- Raw waveform sample: physically direct, but high dimensional and alignment sensitive.
- Mel bin at one time frame: localized perceptual-frequency energy.
- MFCC coefficient: compact combination of log-Mel bands.
- Time-mean MFCC: one number summarizing a whole clip.
- Feature engineering chooses which invariances the model gets for free.

D is small

**Invariance example**
Time averaging makes the representation less sensitive to WHEN a sound occurs—but it also hides temporal order.

tradeoff between dimension D of feature vector $\underline{x}_i$ and the amount of information that is discarded

# Train, validation, and test

Test is used only once to measure accuracy param

THREE DIFFERENT JOBS

≈70%

≈20%

≈10%

20–40 MIN

**TRAIN**
Fit model parameters
Fit data-dependent preprocessing
Seen throughout development

**VALIDATION**
Choose hyperparameters
Compare feature/model choices
May guide development

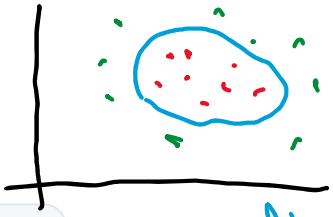
**TEST**
Final unbiased estimate
Do not tune against it
Open after choices are fixed

fit θ in $f(X;\theta)$

choose model

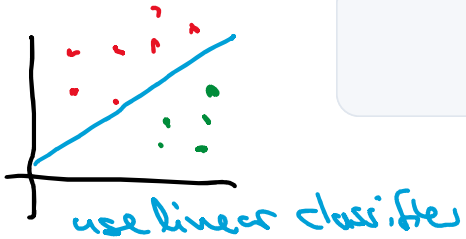
select different hyperparameters

e.g. learning rate



nonlinear kernel

e.g. RBF



use linear classifier

Test set ≠ another validation set

choice of kernel

e.g. linear or RBF or polynomial

# The golden rule for audio: split by source, not just by clip

SEGMENTS FROM ONE PHYSICAL RECORDING ARE CORRELATED

20–40 MIN




**BAD: random clips can place A1 in train and A2 in test**

**GOOD: all clips from a source stay in one split/fold**

Source leakage

clip A.1
clip A.3
training

clip A.2
clip A.3
for testing

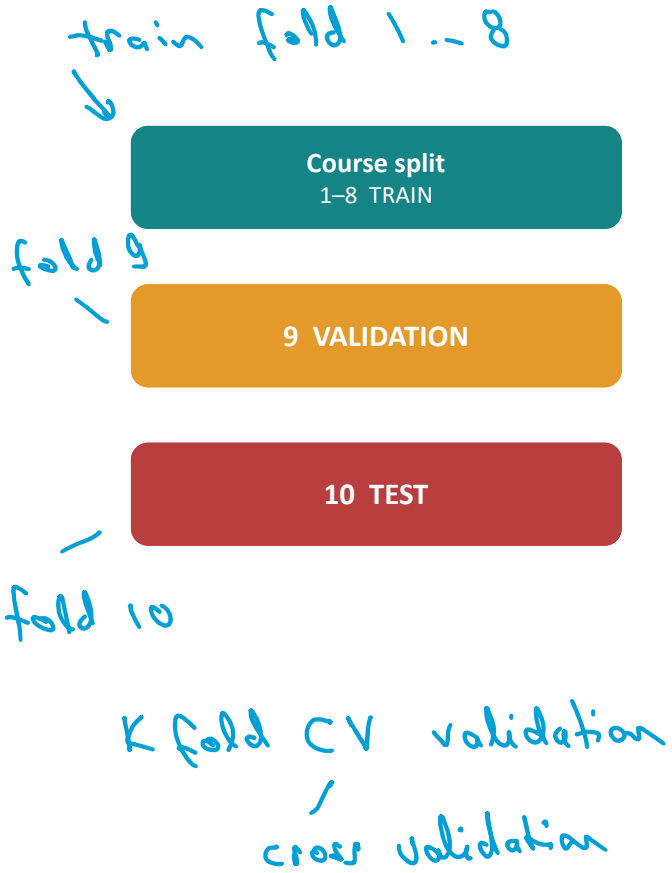
*Instructor cue: Use the siren-segmentation story as a hypothetical. The central problem is source identity leaking across splits.*

# UrbanSound8K: honor the predefined folds

FOLD IDENTITY IS METADATA, NOT A NUISANCE COLUMN

20–40 MIN

- Use the dataset-provided fold assignments instead of blindly calling train_test_split on excerpts.
- For one course baseline, a simple convention is folds 1–8 train, 9 validation, 10 test.
- For stronger estimates, rotate held-out folds and report cross-validation results.
- Keep the split rule fixed while comparing feature representations.



# Leakage can occur in preprocessing too

FIT STATISTICS ON TRAINING DATA ONLY

20–40 MIN

$$z_j = (x_j - \mu_{j,train}) / \sigma_{j,train}$$

- If μ and σ are computed using test clips, information from the test distribution leaks into training.
- scikit-learn Pipeline helps fit transformers only on the training fold during model fitting.
- Random forests do not require standard scaling, but SVM/logistic models often benefit from it.

```
pipe = Pipeline([
('scale', StandardScaler()),
('clf', SVC(kernel='rbf')),
])
pipe.fit(X_train, y_train)
```

normalized data

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

normalized feature

$\mu_j$ is mean of $x_j$ data

$\sigma_j$ is variance of $x_j$ data

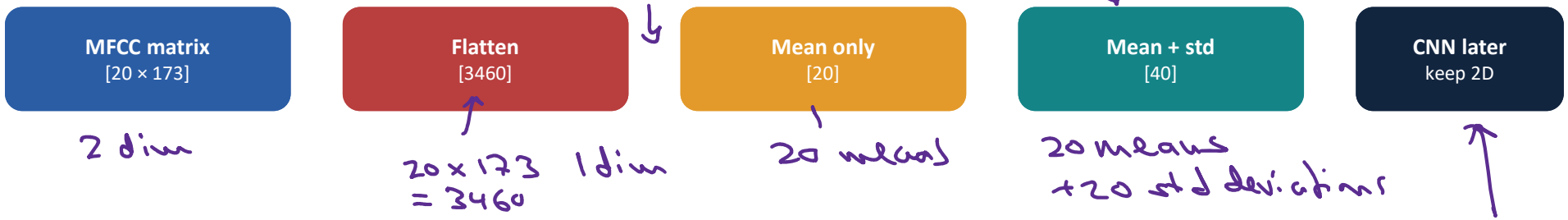
where do $\mu_j$ and $\sigma_j$ come from?

normalize based only on training data

# 2D audio features meet 1D classical models

WE MUST CHOOSE HOW TO REMOVE THE TIME AXIS

40–55 MIN




- Flatten: keeps frame order but creates many coordinates and assumes fixed alignment.
- Mean: very compact but ignores temporal ordering and dynamics.
- Mean + standard deviation: simple compromise for a classical baseline.
- CNNs can consume the 2D map and learn local time–frequency patterns.

waveform $x[n]$ 4 sec at $F_s = 22050$ Hz ⇒ 88200 samples
STFT Nfft = 2048 177 325 features hop ≈ Nfft/4 ⇒ 173 time frames
Mel 64 bands × 173 frames 11072 features
MFCC Matrix 20 coeff 20 × 173 = 3460 features

# Time averaging is a model assumption

WHAT INVARIANCE DID WE JUST HARD-CODE?

40–55 MIN

$$\bar{c}[b] = (1/T)\,\Sigma_t\, C[b,t]$$

- The average keeps "how much" of each coefficient occurred.
- It discards "when," "in what order," and "how long each pattern persisted."
- A rising siren and a falling siren can have similar averages.
- This is exactly the kind of ceiling that motivates sequence models and CNNs.

for classical ML models

**Compression buys simplicity by spending structure.**

# The scikit-learn estimator API

SAME MENTAL MODEL ACROSS MANY ALGORITHMS

55–75 MIN

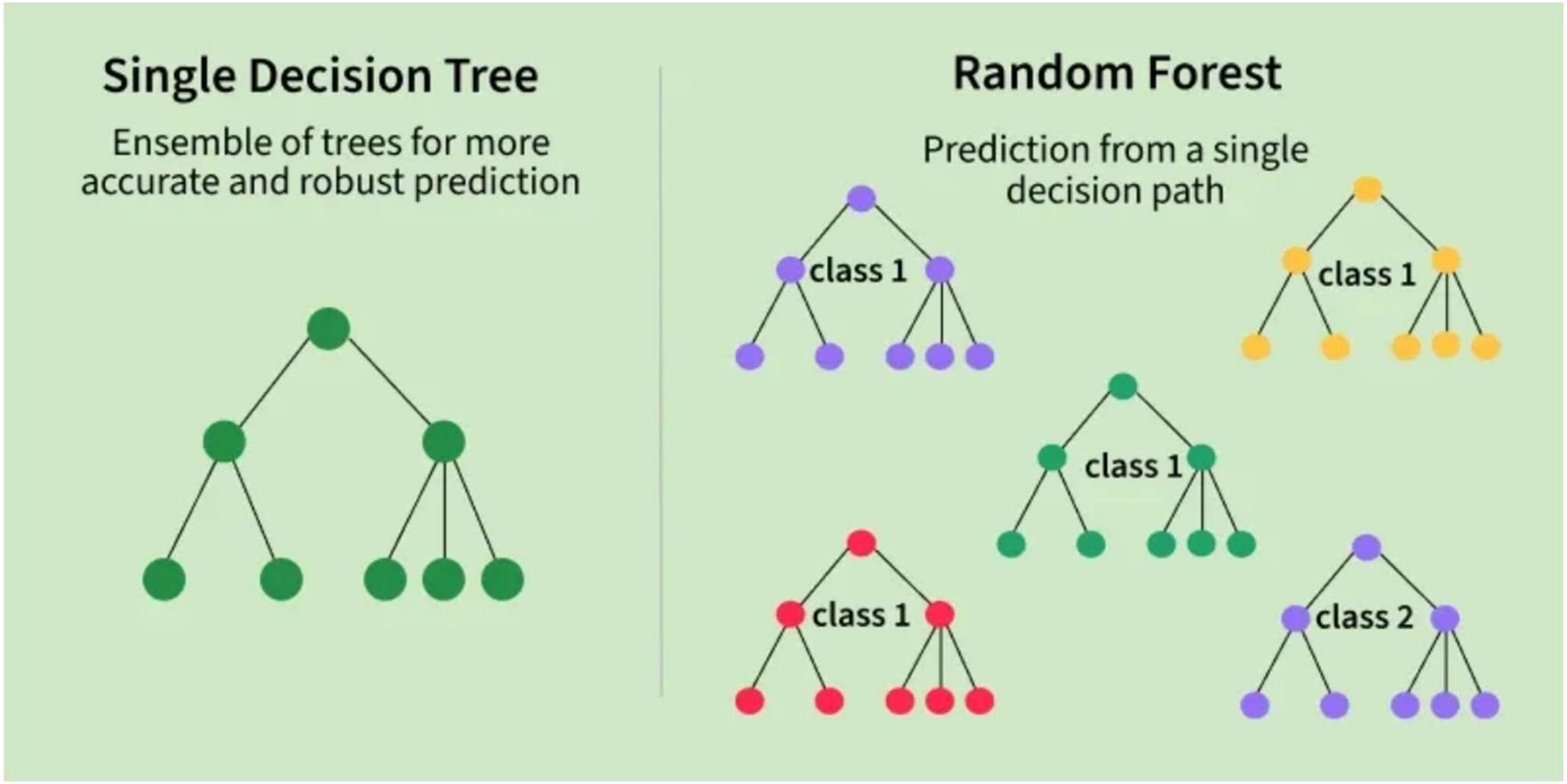
```
clf = RandomForestClassifier(
        n_estimators=300,
        random_state=42,
          n_jobs=-1,
            )

  clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
accuracy = clf.score(X_test, y_test)
```

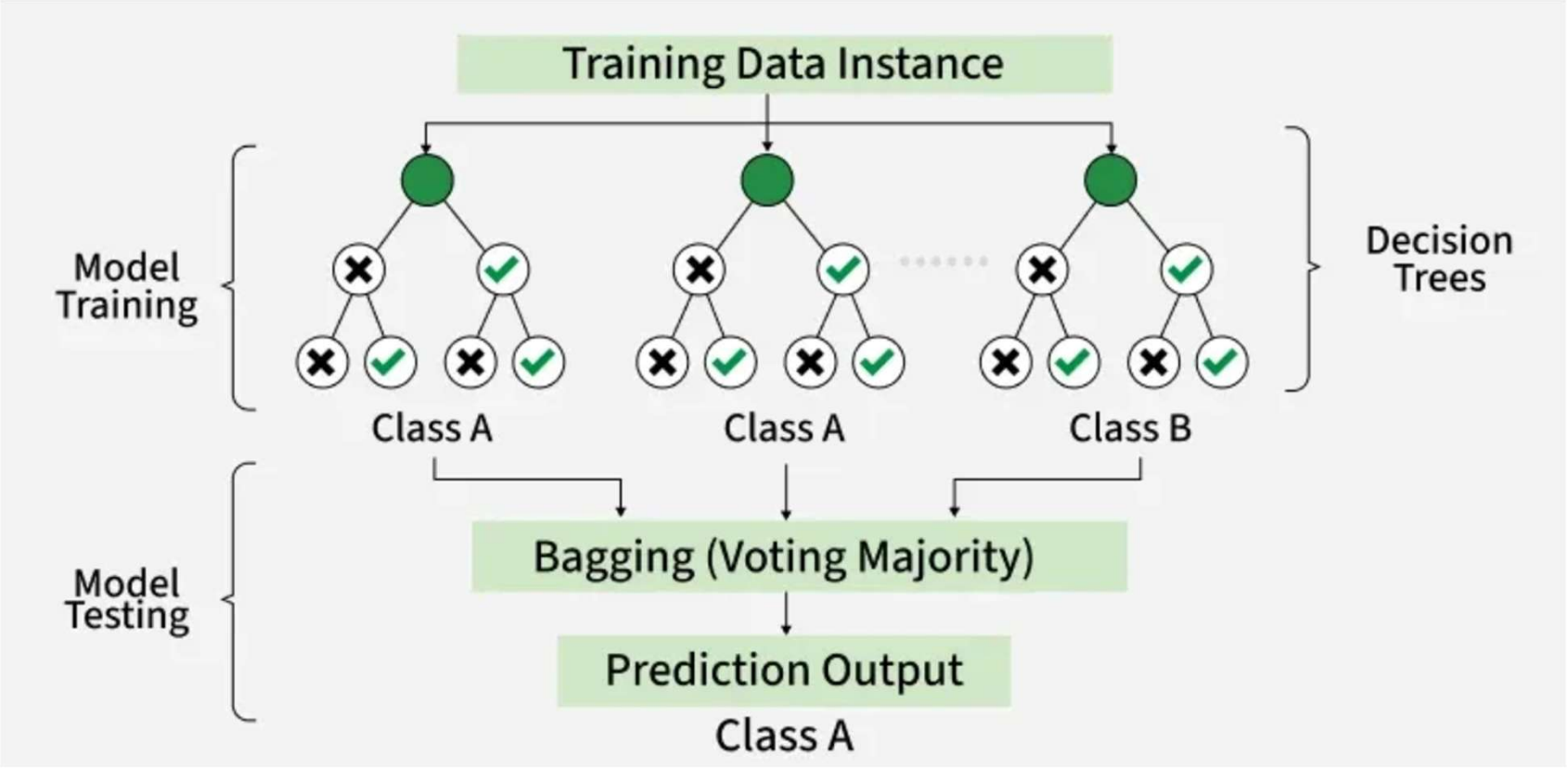
- Instantiate: choose model family and hyperparameters.
- fit: estimate model from training data.
- predict: produce labels for unseen feature vectors.
- score/metrics: quantify performance—then inspect errors.

This is intentionally boring
nice feature of scikit-learn
that you have a simple and
standardized API

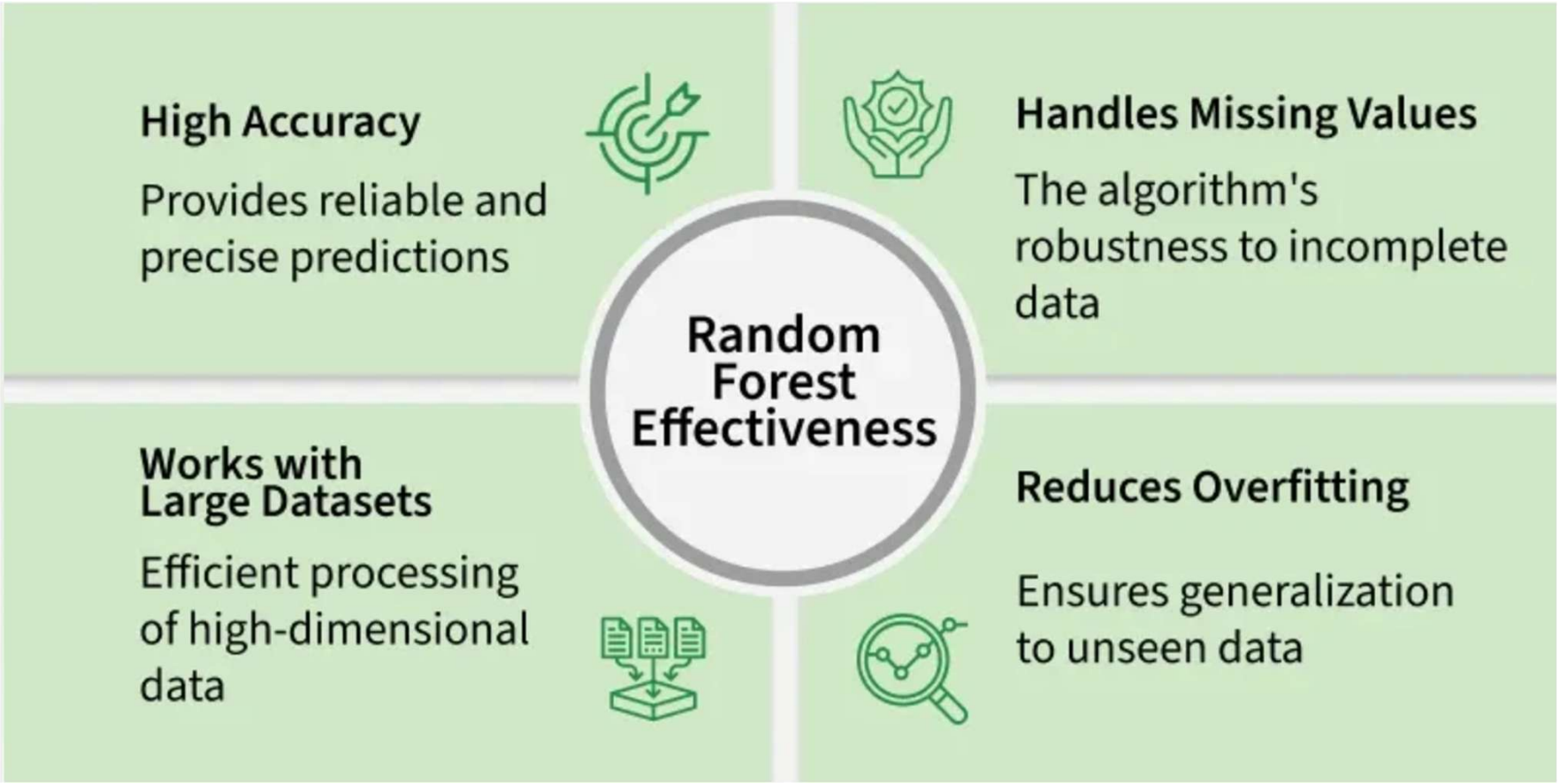
# Random Forest Classifier



# Random Forest Classifier



# Random Forest Classifier



# Why Random Forest is a useful first baseline

NOT NECESSARILY BEST—EXCELLENT FOR DIAGNOSIS

55–75 MIN

- Handles nonlinear decision boundaries without feature scaling.
- Works naturally with compact tabular summary features.
- Fast to train compared with a deep network.
- Provides a sanity check: are the engineered features carrying class information at all?
- Its accuracy is dataset/split/feature dependent—do not promise a fixed 60–65% ceiling.

**Baseline ≠ target**
A baseline should be reproducible, interpretable, and difficult to accidentally leak—not artificially impressive.

## SVC Classifier in Scikit-Learn, Datasets

### Dataset 1

| Label | Signal | Frequency | Noise |
|---|---|---|---|
| Low_tone | Sinusoid | $f = 180 \ldots 320$ Hz | added |
| High_tone | Sinusoid | $f = 1200 \ldots 1700$ Hz | added |
| Chirp | Increasing chirp | $f_0 = 250 \ldots 450$ Hz<br>$f_1 = 2200 \ldots 2800$ Hz | added |

Easy to classify using MFCC_avg and MFCC_std.

### Dataset 2

| Label | Signal | Frequency | Noise |
|---|---|---|---|
| Low_tone | Sinusoid | $f = 180 \ldots 320$ Hz | added |
| High_tone | Sinusoid | $f = 1200 \ldots 1700$ Hz | added |
| Chirp_up | Increasing chirp | $f_0 = 250 \ldots 450$ Hz<br>$f_1 = 2200 \ldots 2800$ Hz | added |
| Chirp_down | Decreasing chirp | $f_0 = 2200 \ldots 2800$ Hz<br>$f_1 = 250 \ldots 450$ Hz | added |

Easy to distinguish low_tone from high_tone, but difficult to distinguish chirp_up from chirp_down using MFCC_avg and MFCC_std.

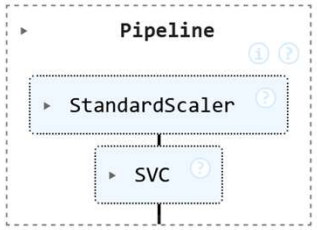
# SVC Classifier in Scikit-Learn, Dataset 1

```
# Prepare train and test feature and label sets
X = np.vstack([mfcc_summary(path, MFCCsplit=MFCCsplit) for path,_ in rows])
y = np.array([label for _,label in rows])
paths = np.array([str(path) for path,_ in rows])
print(f'Dataset: {len(X)} WAV files; X shape={X.shape}; y shape={y.shape}')
print(f'One feature vector: [MFCCmean ({N_MFCC}), MFCCstd ({N_MFCC})]')

X_train, X_test, y_train, y_test, paths_train, paths_test = train_test_split(
    X, y, paths, test_size=0.25, stratify=y, random_state=SEED)
```

```
Dataset: 120 WAV files; X shape=(120, 26); y shape=(120,)
One feature vector: [MFCCmean (13), MFCCstd (13)]
```

```
# Read features, labels, and train model
model = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=3.0, gamma='scale'))
model.fit(X_train, y_train)
```




```
# Prediction on test set
y_pred = model.predict(X_test)
print(f'\nTest accuracy: {accuracy_score(y_test, y_pred):.3f}')
print(f'N_MFCC={N_MFCC}, MFCCsplit={MFCCsplit}')
print('\nClassification report:\n', classification_report(y_test, y_pred, digits=3))
```

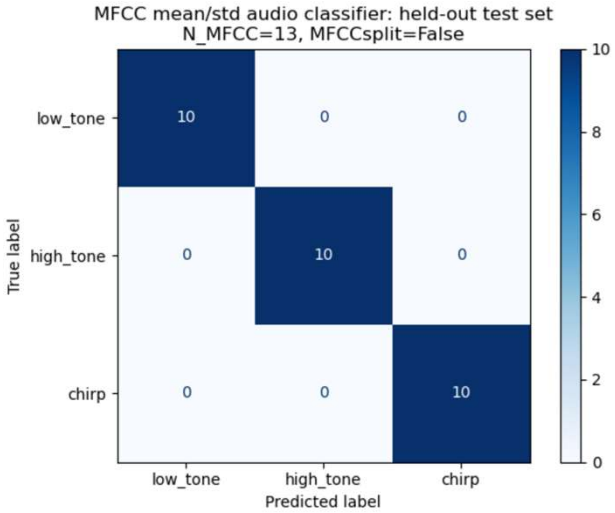
```
Test accuracy: 1.000
N_MFCC=13, MFCCsplit=False

Classification report:
              precision    recall  f1-score   support

       chirp      1.000     1.000     1.000        10
   high_tone      1.000     1.000     1.000        10
    low_tone      1.000     1.000     1.000        10

    accuracy                         1.000        30
   macro avg      1.000     1.000     1.000        30
weighted avg      1.000     1.000     1.000        30
```

```
# Confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, labels=CLASSES, cmap='Blues')
plt.title(f'MFCC mean/std audio classifier: held-out test set\nN_MFCC={N_MFCC}, MFCCsplit={MFCCsplit}')
plt.tight_layout()
plt.show()
```



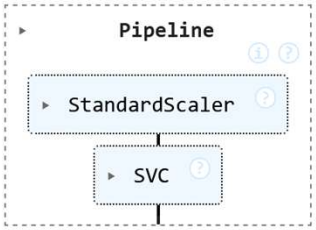
# SVC Classifier in Scikit-Learn, Dataset 2

```
# Prepare train and test feature and label sets
X = np.vstack([mfcc_summary(path, MFCCsplit=MFCCsplit) for path,_ in rows])
y = np.array([label for _,label in rows])
paths = np.array([str(path) for path,_ in rows])
print(f'Dataset: {len(X)} WAV files; X shape={X.shape}; y shape={y.shape}')
print(f'One feature vector: [MFCCmean ({N_MFCC}), MFCCstd ({N_MFCC})]')

X_train, X_test, y_train, y_test, paths_train, paths_test = train_test_split(
    X, y, paths, test_size=0.25, stratify=y, random_state=SEED)
```

```
Dataset: 160 WAV files; X shape=(160, 26); y shape=(160,)
One feature vector: [MFCCmean (13), MFCCstd (13)]
```

```
# Read features, labels, and train model
model = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=3.0, gamma='scale'))
model.fit(X_train, y_train)
```




```
# Prediction on test set
y_pred = model.predict(X_test)
print(f'\nTest accuracy: {accuracy_score(y_test, y_pred):.3f}')
print(f'N_MFCC={N_MFCC}, MFCCsplit={MFCCsplit}')
print('\nClassification report:\n', classification_report(y_test, y_pred, digits=3))
```

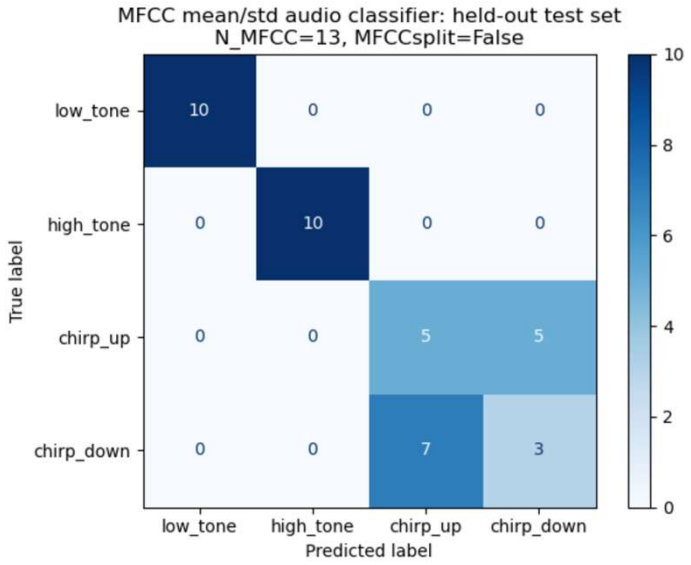
```
Test accuracy: 0.700
N_MFCC=13, MFCCsplit=False

Classification report:
              precision    recall  f1-score   support

  chirp_down      0.375     0.300     0.333        10
    chirp_up      0.417     0.500     0.455        10
   high_tone      1.000     1.000     1.000        10
    low_tone      1.000     1.000     1.000        10

    accuracy                          0.700        40
   macro avg      0.698     0.700     0.697        40
weighted avg      0.698     0.700     0.697        40
```

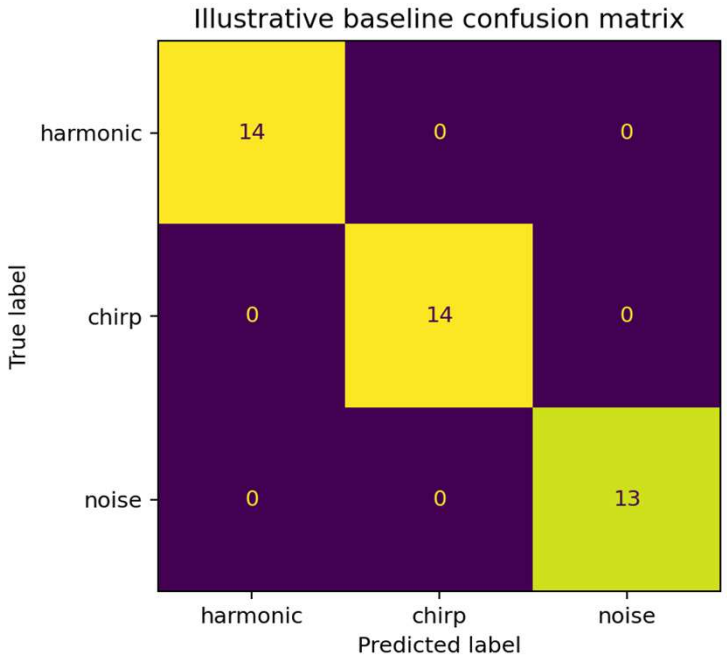
```
# Confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, labels=CLASSES, cmap='Blues')
plt.title(f'MFCC mean/std audio classifier: held-out test set\nN_MFCC={N_MFCC}, MFCCsplit={MFCCsplit}')
plt.tight_layout()
plt.show()
```



# Confusion matrix: accuracy with structure

WHICH CLASSES ARE BEING CONFUSED WITH WHICH?

55–75 MIN




- Rows: true classes; columns: predicted classes in the displayed convention.
- Diagonal cells: correct classifications.
- Off-diagonal cells reveal systematic confusions.
- The example shown is synthetic only—it validates the workflow, not UrbanSound8K performance.

# Accuracy is not the whole evaluation

USE CLASS-SENSITIVE METRICS AND ERROR ANALYSIS

55–75 MIN

$$\text{accuracy} = (\#\ \text{correct}) \,/\, N$$

**Precision**

**Recall**

**Macro-F1**

- Class imbalance can make overall accuracy misleading.
- Macro-averaging gives each class equal weight.
- Inspect representative false positives and false negatives in the original audio.
- Ask whether errors come from the model, the feature representation, or preprocessing.

# Live demo: build X and y without leakage

METADATA DRIVES THE SPLIT

55–75 MIN

```
meta = pd.read_csv(root / 'metadata' / 'UrbanSound8K.csv')

train = meta[meta['fold'].isin(range(1, 9))]
valid = meta[meta['fold'].eq(9)]
test  = meta[meta['fold'].eq(10)]

X_train, y_train = featurize_rows(train, root)
X_valid, y_valid = featurize_rows(valid, root)
X_test,  y_test  = featurize_rows(test, root)
```

*Instructor cue: Ask why splitting metadata first is safer than extracting all clips and shuffling later.*

# A robust baseline feature: MFCC mean + standard deviation

SMALL D, FAST EXPERIMENTS

55–75 MIN

```
mfcc = librosa.feature.mfcc(S=mel_db, n_mfcc=20)
            feat = np.concatenate([
                mfcc.mean(axis=1),
                 mfcc.std(axis=1),
                      ])
             # feat.shape == (40,)
```

- Mean captures typical spectral-envelope coordinates.
- Std captures how much those coordinates vary over time.
- Still destroys sequence order, but usually stronger than mean alone.
- A clean baseline lets next week's CNN earn its complexity.

# Reproducibility is part of engineering

A BASELINE SHOULD BE RERUNNABLE

55–75 MIN

- Freeze the split policy and record feature parameters.
- Set random_state for stochastic estimators.
- Cache extracted features when iteration becomes slow.
- Report class labels, fold assignments, and metrics alongside accuracy.
- Never tune on the test fold because “it is only a classroom project.”

**Reproducibility checklist**
sample rate
duration
n_fft / hop
n_mels / n_mfcc
split folds
model seed

# Error analysis: listen to your mistakes

THE CONFUSION MATRIX TELLS YOU WHERE TO LOOK

55–75 MIN

- Find high-confidence wrong predictions.
- Listen to the clip and inspect its Mel-spectrogram.
- Check label ambiguity, background sounds, clipping, silence, and preprocessing artifacts.
- Compare two representation choices on the same failed example.
- A better feature may fix the problem more cheaply than a bigger model.

**ML engineering loop**
measure → inspect → hypothesize → change one thing → remeasure

# Why can a classical baseline hit a ceiling?

REPRESENTATION MAY BE THE BOTTLENECK

BRIDGE




- Time averaging removes trajectories, modulation patterns, and event shape.
- Flattening keeps order but makes the model sensitive to exact alignment and creates thousands of coordinates.
- A CNN can treat the Mel-spectrogram as a 2D field and learn local patterns that repeat at different times.

**Next week: preserve the picture, learn the filters.**

# Concept check

WHICH EVALUATION IS TRUSTWORTHY?

CHECK

- A. Randomly split all segmented clips, then standardize using the full dataset.
- B. Use source-aware predefined folds; fit preprocessing and model on training only; test once at the end.
- C. Tune hyperparameters until test accuracy stops improving.
- D. Report only training accuracy because the labels are known to be correct.

# Concept check

WHICH EVALUATION IS TRUSTWORTHY?

CHECK

- A. Randomly split all segmented clips, then standardize using the full dataset.
- B. Use source-aware predefined folds; fit preprocessing and model on training only; test once at the end.
- C. Tune hyperparameters until test accuracy stops improving.
- D. Report only training accuracy because the labels are known to be correct.

**Correct: B**

# Lecture 26 synthesis

A TRUSTWORTHY BASELINE IS A SCIENTIFIC INSTRUMENT



- Supervised learning learns $f(x;\theta)$ from labeled feature vectors.
- Data splitting is part of the model: leakage can make an invalid system look excellent.
- Classical models need fixed-length vectors, forcing an explicit choice about time structure.
- scikit-learn makes model APIs consistent; honest evaluation still depends on us.
- The baseline establishes what compact hand-engineered features can do before CNNs see the full 2D representation.

**Exit ticket: Why can a more accurate random clip split be a worse scientific result?**